

第 9, 10 回 分割統治法

執筆: 安永憲司

1 カラツバ法

n ビットの整数 x と y の掛け算 xy は計算量 $O(n^2)$ でできるが, 高速化する方法が知られている. 簡単のため, n は 2 の冪乗だと仮定する. $x = x_1 2^{n/2} + x_2, y = y_1 2^{n/2} + y_2$ を満たすような高々 $n/2$ ビットの非負整数 x_1, x_2, y_1, y_2 が存在する. 単純に掛け算を求めようとする,

$$\begin{aligned} xy &= (x_1 2^{n/2} + x_2) \cdot (y_1 2^{n/2} + y_2) \\ &= x_1 y_1 2^n + (x_1 y_2 + x_2 y_1) 2^{n/2} + x_2 y_2 \end{aligned}$$

という展開から, $n/2$ ビット整数の掛け算を 4 回, $O(n)$ 時間で実行できる足し算やビットシフトを定数回行う必要がある. 足し算やビットシフトは掛け算に比べて低コストであるため, 掛け算の回数に着目する.

ここで, $u = x_1 y_1, v = x_2 y_2, w = (x_1 - x_2)(y_1 - y_2), z = u + v - w$ に対し,

$$\begin{aligned} xy &= x_1 y_1 2^n + (x_1 y_2 + x_2 y_1) 2^{n/2} + x_2 y_2 \\ &= x_1 y_1 2^n + (x_1 y_1 + x_2 y_2 - (x_1 - x_2)(y_1 - y_2)) 2^{n/2} + x_2 y_2 \\ &= u 2^n + (u + v - w) 2^{n/2} + v \end{aligned}$$

と変形できることに注目する. この方法で計算すると, $n/2$ ビット整数の掛け算は 3 回で済む. このアイディアに基づく掛け算は, **カラツバ法** (*Karatsuba algorithm*) と呼ばれる.

```

1  /*****
2  Karatsuba法
3  *****/
4  int karatsuba ( int x, int y ) {
5      int u, v, w, z;
6      // x = x1*2^(n/2) + x2 を満たす (x1, x2) に分ける
7      // y = y1*2^(n/2) + y2 を満たす (y1, y2) に分ける
8
9      u = karatsuba(x1, y1);
10     v = karatsuba(x2, y2);
11     w = karatsuba(x1 - x2, y1 - y2);
12     z = u + v - w;
13
14     // u*(2^n) + z*(2^(n/2)) + v を返す
15 }
```

計算量を比較しよう. n ビット整数の掛け算について, 単純な掛け算の計算量を $T_1(n)$, カラツバ法の計算量 $T_2(n)$ とおく. このとき, n ビット整数の足し算は計算量 $O(n)$ でできるため,

$$T_1(n) = 4T_1(n/2) + O(n), \quad T_2(n) = 3T_2(n/2) + O(n)$$

という漸化式が成り立つ. この漸化式を解くと,

$$T_1(n) = O(n^2), \quad T_2(n) = O(n^{\log_2 3}) = O(n^{1.59})$$

であり、カラツバ法により計算量が改善されている。

2 マスター定理

カラツバ法のように、入力サイズ n に対する問題を、入力サイズのより小さな問題に分割し、小さい問題を再帰的に解いた後に元の問題の解を計算する方法を、**分割統治法** (*divide-and-conquer*) と呼ぶ。

分割統治法によるアルゴリズムの計算量は $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ という形で表現できる。漸化式を解けば計算量が求まるが、その一般的な解法を以下で与える。

定理 2.1 (マスター定理). 整数 $a \geq 1$, 実数 $b > 1$, $d \geq 0$ に対し, $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ が成り立つとき,

$$T(n) = \begin{cases} O(n^d) & d > \log_b a \text{ のとき} \\ O(n^d \log_2 n) & d = \log_b a \text{ のとき} \\ O(n^{\log_b a}) & d < \log_b a \text{ のとき} \end{cases}.$$

証明. 簡単のため, n は b の冪乗と考える. (一般に $\lceil n/b \rceil \leq n/b + 1$ であり, 端数処理によるずれは再帰木の各段階で定数倍の範囲に収まるため, 同じ漸近上界が得られる.) このとき, 入力サイズ n の問題は, $\log_b n$ 回の再帰で自明なケースに到達する. 再帰の枝分かれを木で表すと, 葉の部分 (自明なケース) は深さ $\log_b n$ である. 深さ k のノードは, 入力サイズ n/b^k の部分問題であり, それが合計 a^k 個あるため, その深さの計算量の合計は

$$a^k \cdot O\left(\left(\frac{n}{b^k}\right)^d\right) = O\left(a^k \cdot \left(\frac{n}{b^k}\right)^d\right) = O\left(n^d \cdot \left(\frac{a}{b^d}\right)^k\right)$$

である. これは k についての等比級数であり, $k = 0$ から $k = \log_b n$ までの和が最終的な計算量である.

1. $a/b^d < 1$ ($d > \log_b a$) のとき: 級数は減少し, 初項 $O(n^d)$ が主要項として残る.
2. $a/b^d > 1$ ($d < \log_b a$) のとき: 級数は増加し, 最終項が主要項であり, その値は

$$O(n^d) \left(\frac{a}{b^d}\right)^{\log_b n} = O(n^d) \cdot n^{\log_b(a/b^d)} = O(n^d) \cdot n^{\log_b a - d} = O(n^{\log_b a})$$

である. 指数に関して $x^{\log_b y} = y^{\log_b x}$ という関係を使っている.

3. $a/b^d = 1$ ($d = \log_b a$) のとき: 級数は $O(n^d)$ のままであり, その和は $O(n^d \log_b n) = O(n^d \log_2 n)$.

□

クイズ 2.1. 以下の漸化式を解け. いずれも $T(1) = O(1)$ である.

1. $T(n) = 7T(n/3) + O(n^2)$
2. $T(n) = 9T(n/3) + O(n^2)$
3. $T(n) = 5T(n/3) + O(n)$
4. $T(n) = T(\sqrt{n}) + 1$

クイズ 2.2. 二分探索の計算量に対して成り立つ漸化式におけるマスター定理の定数 (a, b, d) の値は以下のうちどれ.

1. $(a, b, d) = (1, 2, 0)$
2. $(a, b, d) = (1, 2, 1)$
3. $(a, b, d) = (2, 2, 0)$
4. $(a, b, d) = (2, 2, 1)$

クイズ 2.3. 以下の漸化式を解け. いずれも $T(1) = O(1)$ である.

$$1. T(n) = 2T(n/4) + 1$$

$$2. T(n) = 2T(n/4) + \sqrt{n}$$

$$3. T(n) = 2T(n/4) + \sqrt{n}(\log_2 n)^2$$

$$4. T(n) = 2T(n/4) + n$$

$$5. T(n) = 2T(n/4) + n^2$$

3 マージソート

マージソートは、ソートする対象を半分に分割する分割統治法である。

```

1  /*****
2   マージソート
3   入力：配列 A[1...n]
4   出力：ソートされた A[1...n]
5   *****/
6  int[] mergesort ( int A[] ) { // 配列 A[1...n]
7      if ( n <= 1 ) return A;
8      B = mergesort(A[1...n/2]);
9      C = mergesort(A[n/2+1...n]);
10     return merge(B, C);
11 }
12 int[] merge ( int B[], int C[] ) {
13     // B と C の先頭を比較しながら、小さい方から順に出力配列へ移す
14 }
```

関数 merge は、配列 B と C の先頭を比較しながら、小さい方から順に出力配列へ移す。配列の領域が事前に確保されていれば、各要素を定数回処理するだけでよい。B と C の長さをそれぞれ k, m とすると、merge の計算量は $O(k + m)$ である。

関数 mergesort の計算量 $T(n)$ は、

$$T(n) = 2T(n/2) + O(n)$$

を満たすため、マスター定理より、 $T(n) = O(n \log_2 n)$ である。

クイズ 3.1. マージソートの計算量が $O(n \log n)$ になる主な理由として正しいものを選び。

1. 各段階のマージに合計 $O(n)$ 時間かかり、再帰呼び出しの深さが $O(\log n)$ であるため。
2. 各段階のマージに合計 $O(\log n)$ 時間しかかからないため。
3. 再帰呼び出しの回数が $O(\log n)$ 回しかないため。
4. 比較を行わないため。

4 ソート計算量の下界

マージソートが達成した計算量 $O(n \log_2 n)$ は、比較にもとづくソートアルゴリズムとしては最適であることが示せる。つまり、 n 個の要素を比較にもとづいてソートするとき、比較回数が $\Omega(n \log_2 n)$ であることが示せる。

比較にもとづくソートアルゴリズムの計算は、二分木で表現できる。各ノードは $a_i < a_j$ という 1 回の比較に対応し、Yes と No という二つの枝分かれが存在する。葉には、比較結果から定まる要素の大小関

係，すなわち出力すべき順列が対応する．このように，計算の途中で行う判定によって枝分かれする木を**決定木** (decision tree) と呼ぶ．例えば， (a_2, a_3, a_1) という葉は，比較によって $a_2 < a_3 < a_1$ と定まる場合である．このとき， n 個の要素のすべての $n!$ 個の順列が葉に存在する必要がある．もしある順列がないとすれば，そのアルゴリズムではそのように並べ替えることができず，正しくソートできない入力が存在することになる．高さ h の二分木の葉の数は 2^h 以下であり，この数が $n!$ 以上でなければならない．つまり， $2^h \geq n!$ である．一般に，

$$\sqrt{en} \left(\frac{n}{e}\right)^n \leq n! \leq e\sqrt{n} \left(\frac{n}{e}\right)^n$$

が成り立つため，

$$h \geq \frac{1}{2} \log_2(en) + n \log_2(n/e) = \Omega(n \log_2 n)$$

である．高さ h の下界は，比較回数の下界に対応し，それが $\Omega(n \log_2 n)$ であることが示された．

比較にもとづかないソートアルゴリズムである基数ソート (radix sort) では，定数の桁数をもつ整数 n 個を計算量 $O(n)$ でソートできる．

クイズ 4.1. 比較にもとづくソートの決定木で，葉が少なくとも $n!$ 個必要な理由として正しいものを選び．

1. n 個の要素には $n!$ 通りの大小順序があり，それぞれに正しく対応する必要があるため．
2. 1 回の比較で $n!$ 通りの結果に分岐できるため．
3. 高さ h の二分木には必ず $n!$ 個の葉があるため．
4. 比較を $n!$ 回行う必要があるため．

5 クイックソート

クイックソートは最悪時計算量は $O(n^2)$ であるが，実用的には極めて効率的である．特に，異なる n 個の要素に対する (乱択) クイックソートの平均計算量は $O(n \log_2 n)$ であり， $O(\cdot)$ に隠れた定数も小さい．マージソートと異なり，その場アルゴリズム (in-place algorithm) で実現できる．

クイックソートは分割統治法の一つである．配列 $A[p \dots r]$ の要素を並び替え，ピボット $A[q]$ 以下の値をもつ要素を $A[p \dots q-1]$ に，ピボットより大きい値をもつ要素を $A[q+1 \dots r]$ に置く．その後， $A[p \dots q-1]$ と $A[q+1 \dots r]$ のソートを再帰的に行うと，各部分配列はソート済みとなり，全体もソートされる．配列の並び替えによる分割がその場更新で実現できればクイックソート全体もその場更新である．以下のアルゴリズムの `partition` では， $A[r]$ をピボットにして並び替えている．実行の様子は図 1 を参照しよう．

```

1  /*****
2   クイックソート (その場更新版)
3   入力：配列  $A[1 \dots n]$ 
4   出力：ソートされた  $A[1 \dots n]$ 
5   *****/
6  void quicksort ( int A[], int p, int r ) { // 配列  $A[1 \dots n]$  の  $A[p]$  から  $A[r]$  をソートする
7      int q;
8      if ( p < r ) {
9          q = partition( A, p, r );
10         quicksort( A, p, q-1 );
11         quicksort( A, q+1, r );
12     }

```

```

13 }
14 int partition ( int A[], int p, int r ) { // その場更新で配列を並べ替えて分割点qを返す
15     int i, j, tmp, x;
16     x = A[r];          // x をピボットとして選ぶ
17     i = p - 1;
18     for ( j = p; j < r; j++ ) {
19         if ( A[j] <= x ) {
20             i = i + 1;      // i を1増やす
21             tmp = A[i];     // A[i] と A[j] を入れ替える
22             A[i] = A[j];
23             A[j] = tmp;
24         }
25     }
26     tmp = A[i+1];         // A[i+1] と A[r] を入れ替える
27     A[i+1] = A[r];
28     A[r] = tmp;
29     return i+1;
30 }

```

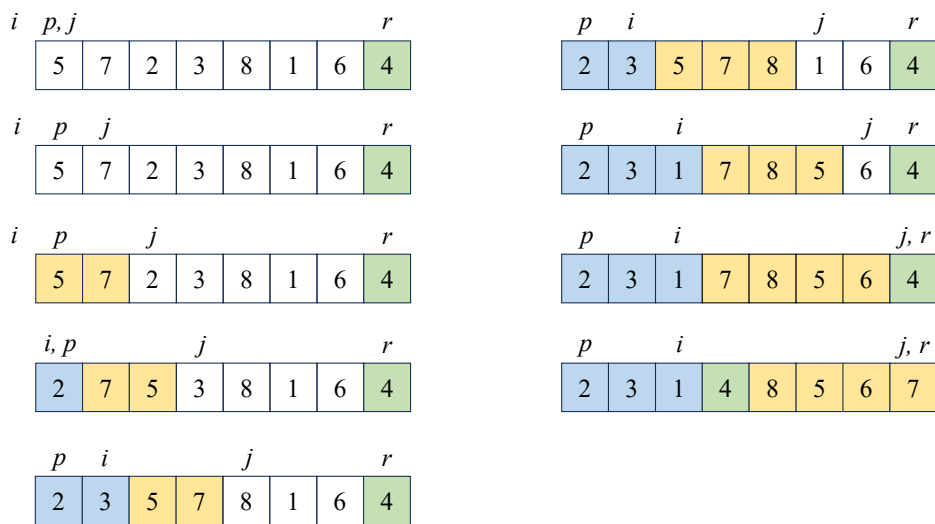


図1 partition の実行の様子

関数 partition の実行の正しさは、以下のループ不変式で確認できる。条件式は、for ループの各繰り返し開始時に、任意のインデックス k に対し、

1. $p \leq k \leq i$ ならば $A[k] \leq x$ (図 1 の青色部分)
2. $i + 1 \leq k \leq j - 1$ ならば $A[k] > x$ (図 1 の黄色部分)
3. $j \leq k \leq r - 1$ の部分はまだ処理していない (図 1 の白色部分)
4. $k = r$ ならば $A[k] = x$ (図 1 の緑色部分)

という 4 条件を満たすことである。

5.1 実行時間の解析

■**最悪時** クイックソートにおいて最も都合が悪いのは、分割したとき、最初に $n - 1$ 個、ピボットの 1 個、残り 0 個と分割されるときである。このときの実行時間 $T(n)$ は、

$$T(n) = T(n - 1) + T(0) + O(n) = T(n - 1) + O(n)$$

であり、 $T(n) = O(n^2)$ である。

■**最良時** 最も都合が良いのは、ちょうど半分ずつに分割できたときであり、その時の実行時間 $T(n)$ は、

$$T(n) = 2T(n/2) + O(n)$$

を満たすため、 $T(n) = O(n \log_2 n)$ である。

■**均衡時** ちょうど半々に分割されなくとも、毎回 9 対 1 の場所で分割されたとすると、

$$T(n) = T(9n/10) + T(n/10) + O(n)$$

である。繰り返し回数は高々 $\log_{10/9}(n) = O(\log_2 n)$ であり、各分割は配列サイズ k に対して $O(k)$ でできるため、全体の実行時間は $O(n \log_2 n)$ である。

5.2 乱択クイックソート

乱択アルゴリズム (randomized algorithm) は、アルゴリズム内でランダムに選んだ値を用いて計算を行うことができ、入力だけでなくランダムに選んだ値に依存して結果が決まる。乱択版のクイックソートでは、ピボットを配列要素からランダムに選ぶ。具体的には、ランダムにインデックス i を選び、 $A[i]$ と $A[r]$ を入れ替えて `partition` を実行すればよい。

補題 5.1. n 個の異なる要素 $z_1 < z_2 < \dots < z_n$ に対する乱択クイックソートにおいて、要素 z_i と z_j ($i < j$) の比較が行われるのは、集合 $Z_{ij} := \{z_i, \dots, z_j\}$ に対し、 z_i と z_j のいずれかがピボットとして Z_{ij} 内で最初に選ばれるときであり、そのときに限る。また、任意の 2 要素の組は高々 1 回しか比較されない。

証明. Z_{ij} 内で最初にピボットとして選ばれたのが x だとする。(1) $z_i < x < z_j$ のとき、 z_i と z_j は比較がされることがない。(2) $x = z_i$ のとき、`partition` により z_i と Z_{ij} のすべての要素が比較される。(3) $x = z_j$ のとき、同様に、 z_j と Z_{ij} のすべての要素が比較される。以上より最初の命題は正しい。また、ピボットとして選ばれた要素は、以後の再帰呼び出しには含まれない。したがって、任意の 2 要素の組は高々 1 回しか比較されない。□

補題 5.2. n 個の異なる要素 $z_1 < z_2 < \dots < z_n$ に対する乱択クイックソートにおいて、要素 z_i と z_j ($i < j$) の比較が行われる確率は $2/(j - i + 1)$ である。

証明. 集合 Z_{ij} において x がピボットとして選ばれる確率は、集合から一様ランダムに選ばれるため、 $1/|Z_{ij}| = 1/(j - i + 1)$ である。補題 5.1 より、 z_i と z_j が比較されるのは、 z_i または z_j がピボットとして Z_{ij} 内で最初に選ばれるときであり、その確率は $2/(j - i + 1)$ である。□

定理 5.3. n 個の異なる要素に対する乱択クイックソートの平均計算量は $O(n \log_2 n)$ である。

証明. n 個の要素 $z_1 < z_2 < \dots < z_n$ に対し,

$$X_{ij} = \mathbb{1}(z_i \text{ と } z_j \text{ が比較される})$$

と定める. ここで, $\mathbb{1}(\cdot)$ は指示関数 (indicator function) であり, 述語 P に対し, P が真であれば $\mathbb{1}(P) = 1$, そうでなければ $\mathbb{1}(P) = 0$ である. 補題 5.1 より, 任意の 2 要素の組は高々 1 回しか比較されないため, クイックソートにおける要素の比較回数 X は,

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}$$

で表される. 期待値の線形性と補題 5.2 を使うと,

$$\begin{aligned} \mathbb{E}[X] &= \mathbb{E} \left[\sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij} \right] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \mathbb{E}[X_{ij}] \\ &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \\ &< \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^{n-1} O(\log_2 n) = O(n \log_2 n). \end{aligned}$$

である. ここで, 調和数について関係式 $\sum_{i=1}^n 1/i = \ln n + O(1)$ を使っている.

クイックソート全体の計算量を考えると, 主な計算は `partition` における比較と要素の入れ替えである. 各 `partition` 呼び出しでは, 1 つのピボットの最終位置が確定し, そのピボットは以後の再帰呼び出しには含まれない. したがって, `partition` の呼び出し回数は高々 n 回である. また, 各 `partition` 呼び出しでは, 対象となる部分配列の各要素を定数回処理する. したがって, クイックソート全体の計算量は $O(n + X)$ である. よって, 乱択クイックソートの平均計算量は

$$O(n + \mathbb{E}[X]) = O(n \log_2 n)$$

である. □

5.3 クイックソートクイズ

クイズ 5.1. クイックソート (その場更新版) に対し, すべて同じ値の要素からなる配列を与えたときの計算量はどれ.

1. $\Theta(n)$ 2. $\Theta(n \log_2 n)$ 3. $\Theta(n^{3/2})$ 4. $\Theta(n^2)$

クイズ 5.2. クイックソート (その場更新版) に対し, 与えられた配列の各要素がすべて異なる値であり, かつ大きい順に並んでいるとき, 計算量は以下のどれだろうか.

1. $\Theta(n)$ 2. $\Theta(n \log_2 n)$ 3. $\Theta(n^{3/2})$ 4. $\Theta(n^2)$

クイズ 5.3. 乱択クイックソートにおいて, ランダムにインデックス i を選ぶ手続きは, 最悪の場合何回実行されるか. また, もっとも良い場合は何回実行されるか.

1. $\Theta(n)$ 2. $\Theta(n \log_2 n)$ 3. $\Theta(n^{3/2})$ 4. $\Theta(n^2)$

6 比較にもとづかないソートアルゴリズム

比較にもとづくアルゴリズムとしては計算量 $O(n \log_2 n)$ が最適であったが，比較にもとづかない方法によりこの下界を回避するアルゴリズムがある．

6.1 計数ソート

計数ソート (counting sort) では， n 個の要素が 0 以上 k 以下の整数であることを仮定し，計算量は $O(n + k)$ である．つまり， $k = O(n)$ であれば計算量 $O(n)$ である．

アイデアとしては，各値 x に対し， x 以下の要素がいくつあるかを数えることである．その数が 10 であれば，値 x をもつ要素の最後の位置は配列の 10 番目である．ただし， x が複数ある場合には同じ位置に置くことはできないため，要素を 1 つ格納するたびに対応する位置を 1 つ前にずらす．

```

1  /*****
2  計数ソート
3  入力：配列  $A[1 \dots n]$ ，配列  $B[1 \dots n]$ ， $n$ ， $k$ 
4  出力： $A$  をソートした結果を配列  $B$  に格納
5  *****/
6  void countingsort ( int A[], int B[], int n, int k ) {
7      int i, j;
8      // 計算用配列  $C[0 \dots k]$  を用意
9      for ( i = 0; i <= k; i++ ) C[i] = 0;           //  $C[i]$  を 0 に初期化
10     for ( j = 1; j <= n; j++ ) C[A[j]] = C[A[j]] + 1; //  $C[i]$  には  $i$  に一致する要素数を格納
11     for ( i = 1; i <= k; i++ ) C[i] = C[i] + C[i-1]; //  $C[i]$  には  $i$  以下の要素数を格納
12     for ( j = n; j > 0; j-- ) { // 安定性を保つため  $A[n]$  から  $A[1]$  の順で結果を  $B$  に格納
13         B[C[A[j]]] = A[j];
14         C[A[j]] = C[A[j]] - 1; // 重複する要素への対処
15     }
16     return;
17 }
```

計算量は $O(n + k)$ であることが確認できる．

計数ソートは**安定性** (stability) と呼ばれる性質をもつ．これは，入力配列において同じ値の要素が，出力配列においても同じ順に登場することであり，次に登場する基数ソートで役に立つ．例えば，グラフを辺数順に並べ替えるとき，同じ辺数をもつグラフ同士の順番が，ソート前と同じ順に保たれるというものである．

6.2 基数ソート

基数ソート (radix sort) は，整数値に対するソートとして使え，一般的には，複数の鍵に関する優先度順で並び替えるときに使える．整数は，各桁毎に 0 から 9 までの値が割り振られており，最上位桁の値が優先度が高く，その次の桁が次に優先度が高いとみなすことができる．

アルゴリズムとしては，安定性をもつソートアルゴリズム (安定ソート) を用いて，優先度の低い鍵から順にソートを行うだけである．整数に対するアルゴリズムの実行の様子 (図 3) を見るとわかりやすい．

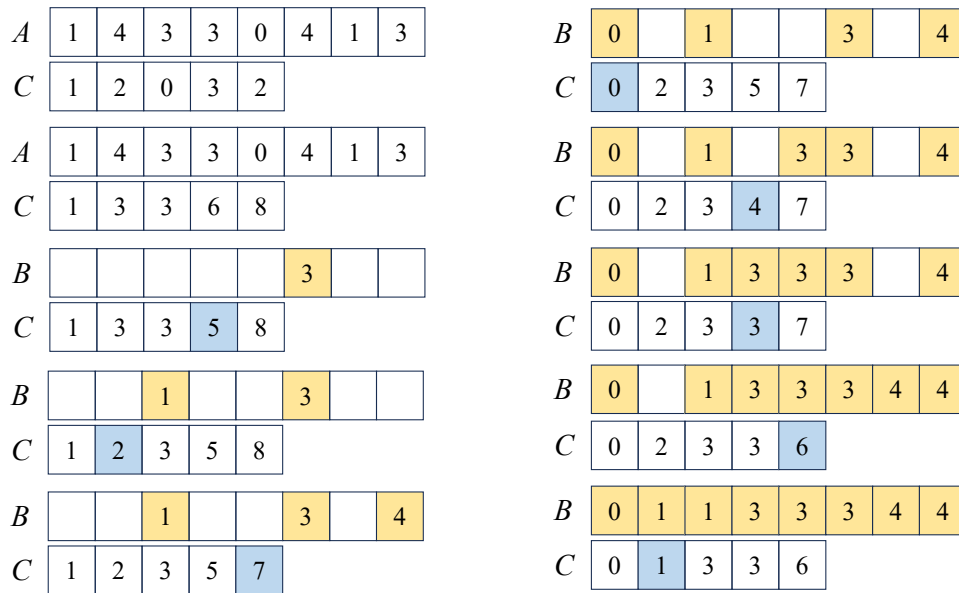


図2 計数ソートの実行の様子

```

1  /*****
2   基数ソート
3   入力：配列 A[1..n], 桁数 d
4   *****/
5  void radixsort ( int A[], int d ) {
6      int i;
7      for ( i = 1; i <= d; i++ ) {
8          安定ソートにより優先度の低い桁から順に配列Aをソートする
9      }
10     return;
11 }

```

2324	4512	4512	2315	1315
1324	3532	4513	1315	1324
2315	4513	3513	2324	2315
1315	3513	2315	1324	2324
4512	2324	1315	4512	3513
3532	1324	2324	4513	3532
4513	2315	1324	3513	4512
3513	1315	3532	3532	4513

図3 基数ソートの実行の様子

計算量は、 n 個の入力の各桁が k 通りの値をとるとき、計数ソートにより安定ソートを計算量 $O(n + k)$ で実現できるため、基数ソートは計算量 $O(d(n + k))$ でできる。

6.3 ソートクイズ

クイズ 6.1. 計数ソートが効率的に動作するための条件として正しいものをすべて選べ.

1. 入力の値域 k が入力サイズ n に比べて十分小さい
2. 入力が整数である
3. 入力がすでにソートされている
4. 入力に重複がない

クイズ 6.2. 基数ソートで各桁のソートに安定なソートを使わなかった場合、どのような問題が生じるか.

1. 計算量が増加する
2. ソート結果が正しくなくなる
3. メモリ使用量が増加する
4. 入力が整数でないと動作しない

7 中央値とクイック選出

n 個の要素の中央値とは、 n 個の真ん中に位置する値 (50 パーセンタイル) であり、 n が奇数なら一意に定まる. 中央値を求める一つの方法は、 n 個の要素をソートしてその真ん中を選ぶことであり、計算量は $O(n \log_2 n)$ である. ここでは $O(n)$ で計算する方法を見てみる.

中央値よりも一般的な問題として**選出問題** (*selection problem*) を考える. 入力は要素の集合 S と整数 k であり、 S において小さい方から k 番目の要素を出力するという問題である.

分割統治法を用いて計算する方法がある. このアルゴリズムは**クイック選出・クイックセレクト** (*quick select*) と呼ばれる. 値 v に対し、集合 S を、値が v より小さい集合 S_L 、 v と同じ集合 S_v 、値が v より大きい集合 S_R に分割する. 選出問題を解く関数 $\text{Select}(S, k)$ は、以下のように小問題に分割できる.

$$\text{Select}(S, k) = \begin{cases} \text{Select}(S_L, k) & k \leq |S_L| \text{ のとき} \\ v & |S_L| < k \leq |S_L| + |S_v| \text{ のとき} \\ \text{Select}(S_R, k - |S_L| - |S_v|) & k > |S_L| + |S_v| \text{ のとき} \end{cases}$$

分割する集合 S_L, S_v, S_R は線形時間で計算でき、その場更新で実現できる. 上記アルゴリズムは、 v の選び方以外は定まっている. もし、値 v を小さい計算量で選ぶことができ、かつ $|S_L| \approx |S|/2, |S_R| \approx |S|/2$ を満たせば、計算量は $T(n) = T(n/2) + O(n)$ であり、線形時間で選出問題を解くことができる. ただ、これを満たす v を選ぶことは、中央値を選ぶことに近く、中央値の計算のために中央値を選ぶことになっている.

そこで、 v を S からランダムに選ぶことにする. 直感的には、 v が中央値に近いほど部分問題は小さくなる. 例えば、 v が現在の集合の 25 パーセンタイルと 75 パーセンタイルの間にあれば、 $|S_L|$ と $|S_R|$ は $(3/4)|S|$ 以下である. 以下では、期待計算量を直接評価する.

図 4 は、クイック選出の実行の様子を示している. ピボットである v をランダムに選び、その後、 S_L または S_R を選ぶことで問題のサイズを小さくしている.

サイズ n の S に対するクイック選出の期待計算量を $T(n)$ とする. ピボットが小さい方から i 番目の要素である確率は $1/n$ である. 探す順位 k によって再帰先は異なるが、そのサイズは高々 $\max\{i-1, n-i\}$

23	5	12	8	31	7	18	14	2	26	9	4	30	11	19
5	2	4	7	23	12	8	31	18	14	26	9	30	11	19
5	2	4	7	23	12	8	18	14	9	11	19	26	31	30
5	2	4	7	12	8	14	9	11	18	23	19	26	31	30
5	2	4	7	8	9	12	11	14	18	23	19	26	31	30
5	2	4	7	8	9	11	12	14	18	23	19	26	31	30

図4 クイック選出の実行の様子

である。したがって、

$$T(n) \leq \frac{1}{n} \sum_{i=1}^n T(\max\{i-1, n-i\}) + O(n)$$

である。右辺の和では、同じ大きさの部分問題は高々2回ずつ現れるため、

$$T(n) \leq \frac{2}{n} \sum_{m=\lceil n/2 \rceil}^{n-1} T(m) + O(n)$$

である。

この漸化式から $T(n) = O(n)$ が従う。実際、 $O(n)$ の部分を Cn で上から抑える。十分大きい定数 $c > 0$ に対し、すべての $m < n$ で $T(m) \leq cm$ が成り立つと仮定すると、

$$T(n) \leq \frac{2}{n} \sum_{m=\lceil n/2 \rceil}^{n-1} cm + Cn \leq \frac{2c}{n} \cdot \frac{3n^2}{8} + Cn = \frac{3c}{4}n + Cn.$$

$c \geq 4C$ となるように選べば、 $T(n) \leq cn$ である。よって帰納法により $T(n) = O(n)$ である。

選出問題を、決定性アルゴリズムによって計算量 $O(n)$ で解く方法も知られている。それは**中央値の中央値法** (median of medians) と呼ばれる。

7.1 クイック選出クイズ

クイズ 7.1. クイック選出アルゴリズムの最悪計算量が $O(n^2)$ となる入力例として正しいものを選べ。

1. 毎回中央値をピボットに選ぶ場合
2. 毎回最小値または最大値をピボットに選ぶ場合
3. 入力がすべて同じ値の場合
4. 入力がランダムな場合

クイズ 7.2. クイックソートとクイック選出に関する次の記述のうち、正しいものをすべて選べ。

1. クイックソートは安定なソートアルゴリズムであるが、クイック選出は安定性の概念が適用されない。

- クイック選出における最悪計算量 $O(n^2)$ は、ピボットの選択戦略が偏ることによって発生する。
- 中央値の中央値法は、クイック選出の最悪計算量を線形時間 $O(n)$ に保証する決定性アルゴリズムである。
- 毎回中央値をピボットとして選べば、クイックソートの最悪計算量は $O(n \log n)$ になる。
- 末尾要素をピボットにする上記のクイックソートでは、配列がすでに整列済みのとき、最悪計算量 $\Theta(n^2)$ となる。

8 行列の掛け算・シュトラッセン法

$n \times n$ 行列 X と Y の積 $Z = XY$ を計算したいとき、その (i, j) 成分は、

$$Z_{ij} = \sum_{k=1}^n X_{ik} Y_{kj}$$

である。行列の要素同士の演算計算量が $O(1)$ のとき、単純に計算すれば、上記の一つの成分の計算は計算量 $O(n)$ であり、 Z 全体では $O(n^3)$ がかかる。

この計算量を分割統治法によって改善することができる。行列 X, Y を $n/2 \times n/2$ のブロックに分割する。つまり、

$$X = \begin{bmatrix} A & B \\ C & D \end{bmatrix}, \quad Y = \begin{bmatrix} E & F \\ G & H \end{bmatrix}.$$

このとき、行列積の計算は、

$$XY = \begin{bmatrix} A & B \\ C & D \end{bmatrix} \begin{bmatrix} E & F \\ G & H \end{bmatrix} = \begin{bmatrix} AE + BG & AF + BH \\ CE + DG & CF + DH \end{bmatrix}$$

と書ける。つまり、 $n \times n$ 行列の積は、 $n/2 \times n/2$ の行列積 8 回と、足し算の計算量 $O(n^2)$ で計算できる。つまり、計算量 $T(n)$ は、

$$T(n) = 8T(n/2) + O(n^2)$$

である。この漸化式を解くと $T(n) = O(n^3)$ である。手順を工夫することで、 $n/2 \times n/2$ の行列積 7 回で計算する方法がある。これはシュトラッセン (Strassen) 法と呼ばれ、

$$\begin{aligned} P_1 &= A(F - H) & P_2 &= (A + B)H & P_3 &= (C + D)E & P_4 &= D(G - E) \\ P_5 &= (A + D)(E + H) & P_6 &= (B - D)(G + H) & P_7 &= (A - C)(E + F) \end{aligned}$$

に対し、

$$XY = \begin{bmatrix} P_5 + P_4 - P_2 + P_6 & P_1 + P_2 \\ P_3 + P_4 & P_1 + P_5 - P_3 - P_7 \end{bmatrix}$$

と計算する。計算量 $T(n)$ は、

$$T(n) = 7T(n/2) + O(n^2)$$

であり、マスター定理より、 $T(n) = O(n^{\log_2 7}) = O(n^{2.81})$ である。

8.1 シュトラッセン法クイズ

クイズ 8.1. 複素数 $a + bi$ と $c + di$ の積 $(ac - bd + (ad + bc)i)$ を、実数の掛け算 3 回で計算する方法を示せ。

クイズ 8.2. シュトラッセンのアルゴリズムに関する説明として正しいものをすべて選べ.

1. 行列積の計算回数を減らすことで高速化を実現している
2. 2×2 行列の積を 7 回の掛け算で計算できる
3. 標準的な行列積アルゴリズムよりも理論上の計算量が小さい
4. 任意のサイズの行列に対して常に最速である

9 分割統治法の演習問題

演習問題 9.1. 長さ n の数列が単峰性をもつ (unimodal) とは, ある値まで数列は増加し, その後減少するときである. 単峰性をもつ長さ n の数列が与えられたとき, その最大値を $O(\log_2 n)$ で求めるアルゴリズムを設計せよ.

演習問題 9.2. 異なる整数を格納しているソート済み配列 $A[1 \dots n]$ が与えられたとき, $A[i] = i$ を満たす i が存在するかどうかを判定する最速のアルゴリズムを設計せよ.

演習問題 9.3. アルゴリズムの計算量 $T(n)$ に関する漸化式が, $T(n) \leq T(\lfloor \sqrt{n} \rfloor) + 1$ を満たすとき, 計算量の正しい上界としてもっとも小さいものを選べ.

1. $O(1)$
2. $O(\log_2 \log_2 n)$
3. $O(\log_2 n)$
4. $O(\sqrt{n})$

演習問題 9.4. 乱択クイックソートの計算量解析を, 各再帰呼び出しの平均計算量に着目して行う. 与えられる n 個の入力はすべて異なる値だとする. サイズ n の入力に対する期待計算量を $T(n)$ とする.

1. ピボットが i 番目に小さい要素である確率を求めよ.
2. 次の漸化式が成り立つことを示せ.

$$T(n) = \frac{1}{n} \sum_{i=1}^n (T(i-1) + T(n-i)) + \Theta(n).$$

3. 上記より,

$$T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + \Theta(n)$$

を示せ.

4. $n \geq 2$ に対し,

$$\sum_{i=1}^{n-1} i \log_2 i \leq \frac{n^2}{2} \log_2 n - \frac{n^2}{8}$$

を示せ. (和を $i = 1, \dots, \lceil n/2 \rceil - 1$ と $i = \lceil n/2 \rceil, \dots, n-1$ の 2 つに分けてみよう.)

5. 上記の結果を用いて, $T(n) = O(n \log_2 n)$ であることを示せ.

演習問題 9.5. 入力 $0, 2, 0, 1, 3, 4, 6, 1, 3, 2$ に対し, 計数ソートを実行する様子を図示せよ.

演習問題 9.6. 計数ソートの最後のループの正しさを, 以下のループ不変式を用いて示せ. 最後のループの各繰り返しの開始時, 任意の値 v に対し, まだ B に格納されていない値 v の要素が存在するならば, 次に値 v を格納すべき位置は $B[C[v]]$ である.

演習問題 9.7. 基数ソートを用いて、以下の文字列を辞書順にソートする様子を示せ.

とうち, そうと, ぴぼと, ひかく, きすう, ひいぷ, りすと, きゆう

演習問題 9.8. 以下のソートアルゴリズムのうち安定性をもつものをすべて選べ.

1. 挿入ソート
2. マージソート
3. ヒープソート
4. クイックソート

さらに、比較に基づくソートアルゴリズムを安定にする方法を示し、追加の時間計算量と空間計算量を示せ.

演習問題 9.9. 非負整数が格納された任意の配列 $A[1\dots n]$ に対し、

$$M := \max_i A[i] - \min_i A[i]$$

としたとき、計算量 $O(n + M)$ でこの配列をソートするアルゴリズムを示せ.

演習問題 9.10. 実数 $x_1, \dots, x_n \in \mathbb{R}$ を考える.

1. $x_1, \dots, x_n$ の中央値 μ_1 は関数

$$\sum_{i=1}^n |x_i - \mu|$$

を最小化する μ であることを示せ. n は奇数と仮定してよい.

2. $x_1, \dots, x_n$ の平均値 μ_2 は関数

$$\sum_{i=1}^n (x_i - \mu)^2$$

を最小化する μ であることを示せ.

演習問題 9.11. 配列 $A[1\dots n]$ に多数要素があるとは、半分より多くの要素が同じときである. 配列が与えられ、多数要素があるかどうかを判定し、もしあればその要素を出力するアルゴリズムを設計したい. ただし、配列の要素は数値のような順序集合とは限らない. ($A[i] < A[j]$ のような比較はできないが、 $A[i] = A[j]$ という判定はできる.)

1. 計算量 $O(n \log_2 n)$ で解くアルゴリズムを与えよ. (ヒント: 配列 A を同じサイズの配列 A_1, A_2 に分割したとき、 A_1, A_2 の多数要素を知っていることが、 A の多数要素を知ることに役立つか考えよう.)
2. 計算量 $O(n)$ で解くアルゴリズムを与えよ. (ヒント: 配列 A の要素から $n/2$ 個のペアを作り、各ペアは違う値なら捨て、同じ値なら残す. この操作により、高々 $n/2$ 要素が残り、もし A に多数要素があれば残った配列にも多数要素があることを示そう.)

演習問題 9.12. 行列 A の平方 (square) とは、 AA のことである.

1. 2×2 行列の平方の計算には掛け算 5 回で十分であることを示せ.
2. $n \times n$ 行列の平方計算に関する以下の記述の間違いを指摘せよ.

シュトラッセン法と同様に分割統治法を用いる. シュトラッセン法はサイズ $n/2$ の部分問題を 7 回解くが、1 で示した事実より、サイズ $n/2$ の部分問題を 5 回解けば十分である. したがって、アルゴリズムの計算量は $O(n^{\log_2 5})$ である.

3. 実際は、平方計算は掛け算よりも簡単ではない. ここでは、 $n \times n$ 行列の平方が計算量 $S(n) = O(n^c)$ でできるとき、任意の $n \times n$ 行列の積が計算量 $O(n^c)$ でできることを示す. ただし、 $c \geq 2$ である.
- (a) $n \times n$ 行列 A, B に対し、 $AB + BA$ は計算量 $3S(n) + O(n^2)$ でできることを示せ.
- (b) $n \times n$ 行列 X, Y に対し、 $2n \times 2n$ 行列 A, B を

$$A = \begin{bmatrix} X & 0 \\ 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & Y \\ 0 & 0 \end{bmatrix}$$

と定める. $AB + BA$ を X と Y を用いて表せ.

- (c) 上記 (a) と (b) を用いて、 $n \times n$ 行列の積 XY が計算量 $3S(2n) + O(n^2)$ でできることを示せ. 結果として、行列積の計算量は $O(n^c)$ である.

クイズの答え

クイズ 2.1 1. $O(n^2)$

2. $O(n^2 \log_2 n)$

3. $O(n^{\log_3 5})$

4. $O(\log_2 \log_2 n)$

クイズ 2.2 1. $(a, b, d) = (1, 2, 0)$.

クイズ 2.3 1. $O(\sqrt{n})$

2. $O(\sqrt{n} \log_2 n)$

3. $O(\sqrt{n}(\log_2 n)^3)$

4. $O(n)$

5. $O(n^2)$

クイズ 3.1 1. 各段階のマージに合計 $O(n)$ 時間かかり、再帰呼び出しの深さが $O(\log n)$ であるため.

クイズ 4.1 1. n 個の要素には $n!$ 通りの大小順序があり、それぞれに正しく対応する必要があるため.

クイズ 5.1 4. $\Theta(n^2)$.

クイズ 5.2 4. $\Theta(n^2)$.

クイズ 5.3 最悪の場合も、もっとも良い場合も 1. $\Theta(n)$.

クイズ 6.1 1, 2.

クイズ 6.2 2. ソート結果が正しくなくなる.

クイズ 7.1 2. 毎回最小値または最大値をピボットに選ぶ場合.

クイズ 7.2 2, 3, 4, 5.

クイズ 8.1 例えば,

$$p_1 = ac, \quad p_2 = bd, \quad p_3 = (a+b)(c+d)$$

とおくと,

$$(a+bi)(c+di) = (p_1 - p_2) + (p_3 - p_1 - p_2)i$$

と計算できる.

クイズ 8.2 1, 2, 3.